BABU BANARASI DAS UNIVERSITY

SCHOOL OF COMPUTER APPLICATIONS

NoSQL PROJECT REPORT

STUDENT MANAGEMENT SYSTEM USING MONGODB

MongoDB • NoSQL • CRUD • Query Operators • Sorting • Filtering

Submitted To

Mr. Harendra Singh Rajpoot

Submitted By

Jijivisha Tiwari

Department

BCA (Data Science & Artificial Intelligence)

Roll No.

66

Class

BCA DS & AI 23

Academic Session

2026–2027

Project based on the supplied MongoDB project reference document and execution screenshots.

PROJECT OVERVIEW

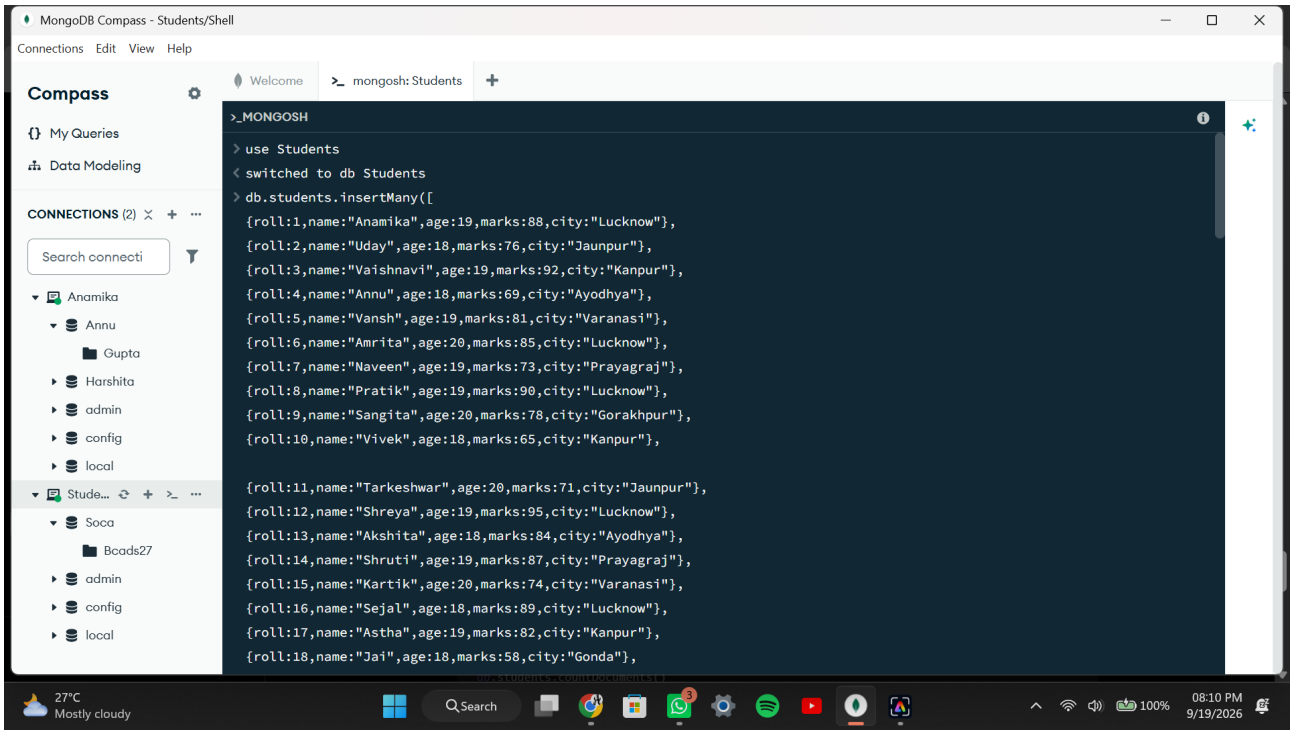
The project models a simple student information system. The MongoDB database is named **Students** and the collection is named **students**. Each student document contains five main fields: roll, name, age, marks and city.

The supplied reference contains the complete 50-student dataset and embedded screenshots showing MongoDB Compass / mongosh operations. The screenshots are used throughout this report in place of reproducing the MongoDB commands as plain text.

Field	Purpose	Example
roll	Unique student roll number	1
name	Student name	Anamika
age	Student age	19
marks	Student marks	88
city	Student city	Lucknow

1. CREATE DATABASE

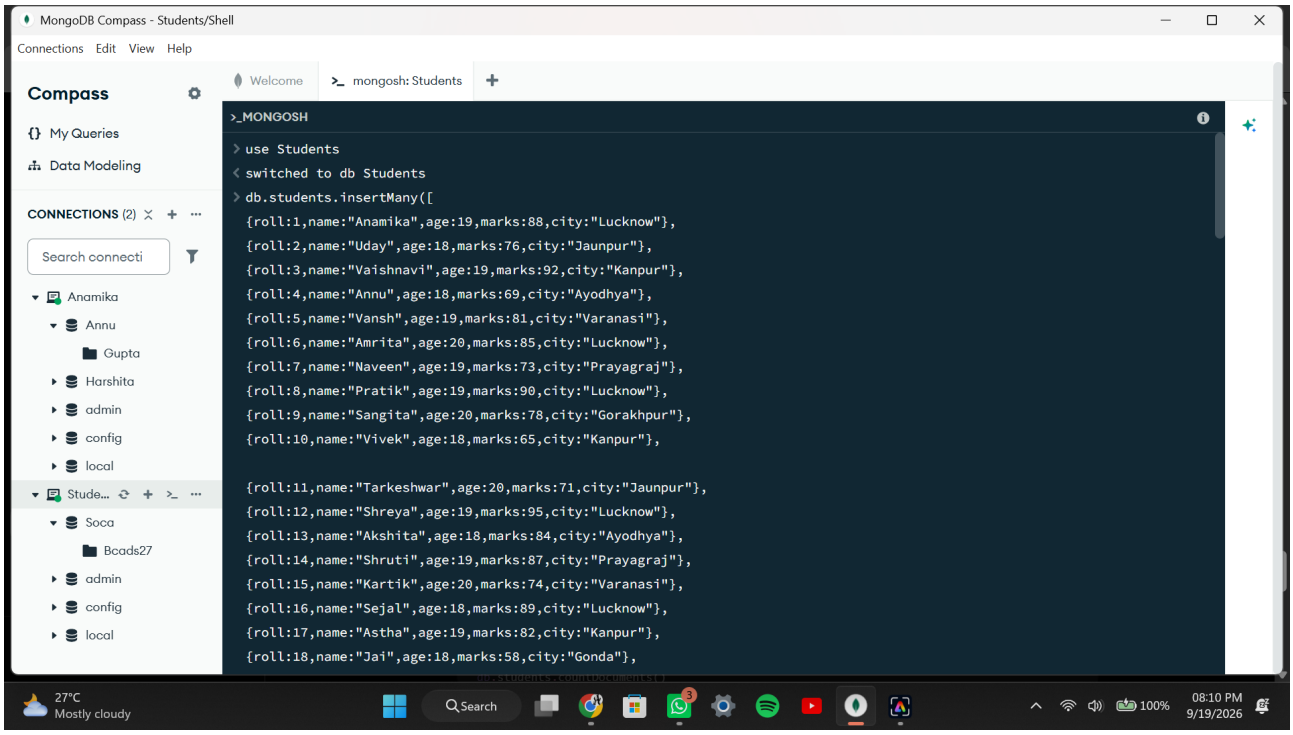
The project uses the MongoDB database named “Students”. The supplied screenshot shows the MongoDB shell switching to the Students database.



Screenshot 1: MongoDB Compass shell showing the Students database and project session.

2. CREATE COLLECTION

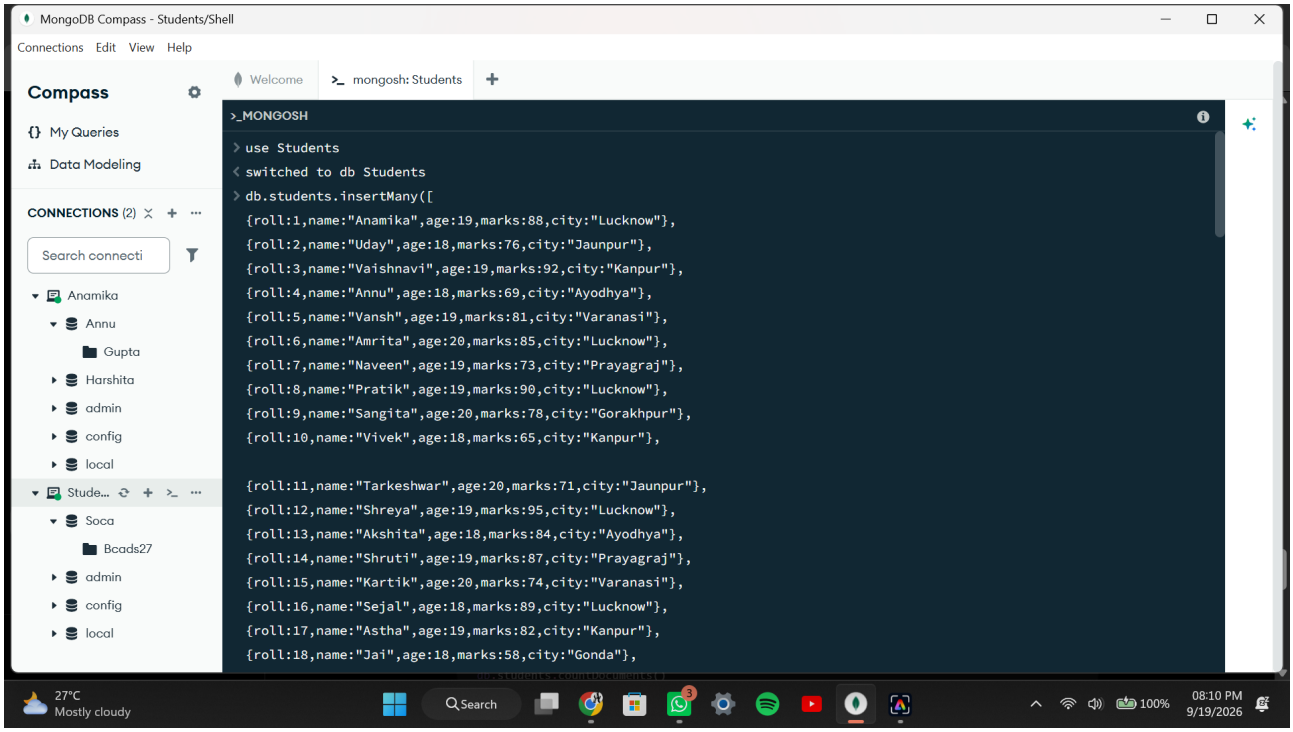
The student records are stored in a collection named “students”. The reference screenshot shows the collection context used for the project.



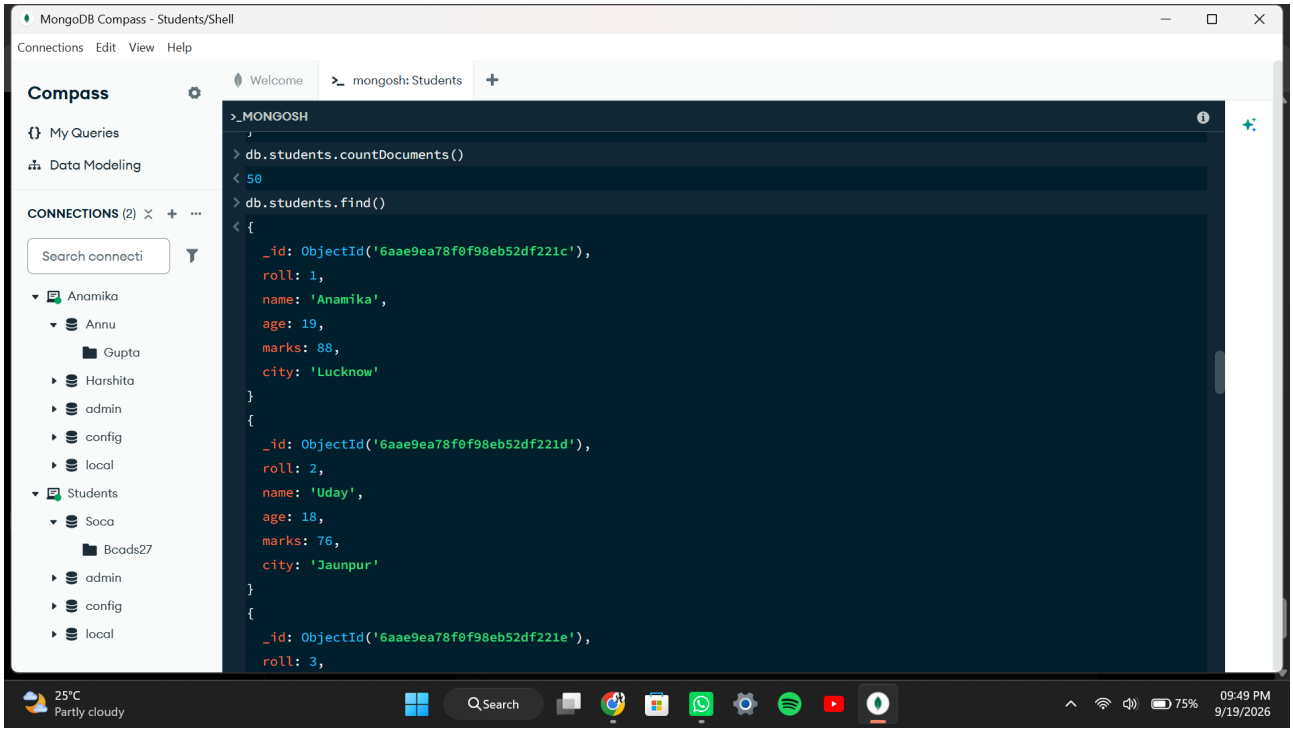
Screenshot 2: Students database session used for the student collection.

3. INSERT 50 STUDENTS

The reference project inserts 50 student documents. Each document contains roll number, name, age, marks and city. The embedded screenshots show the insertion session and the student data being entered.



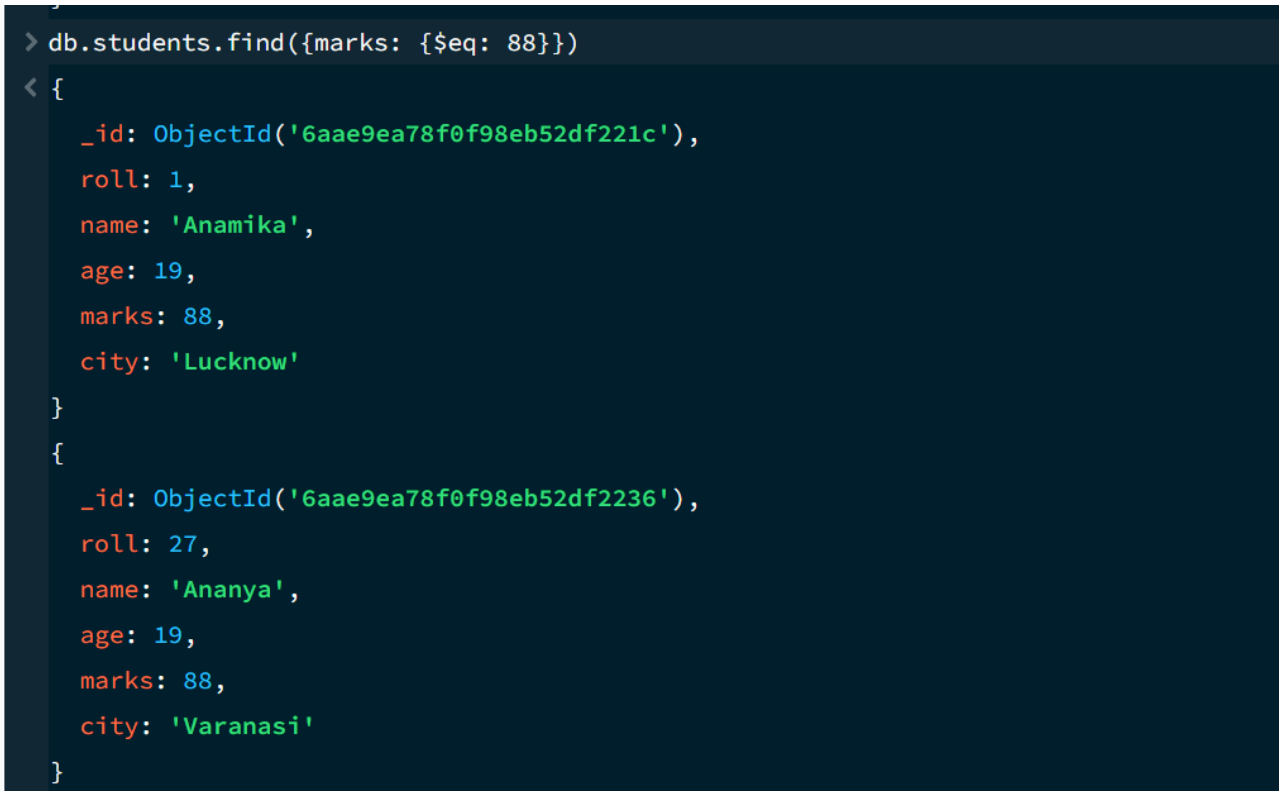
Screenshot 3: First portion of the 50-student insertion data.



Screenshot 4: Continued MongoDB student insertion/data session.

4. DISPLAY ALL STUDENTS

The display operation retrieves documents from the students collection. The reference contains a MongoDB result screenshot for retrieving student records.



Screenshot 5: MongoDB result showing student documents.

5. EQUAL TO — $eq

The $eq comparison operator is used to find students whose marks are exactly 88.

```
> db.students.find({marks: {$gt: 85}})
< {
    _id: ObjectId('6aae9ea78f0f98eb52df221c'),
    roll: 1,
    name: 'Anamika',
    age: 19,
    marks: 88,
    city: 'Lucknow'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df221e'),
    roll: 3,
    name: 'Vaishnavi',
    age: 19,
    marks: 92,
    city: 'Kanpur'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df2223'),
    roll: 8,
    name: 'Pratik',
    age: 19,
    marks: 90,
    city: 'Lucknow'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df2227'),
    roll: 12,
    name: 'Shreya',
    age: 19,
    marks: 95,
    city: 'Lucknow'
  }
```

Screenshot 6: MongoDB query/result for the equal-to comparison.

6. GREATER THAN — $gt

The $gt comparison operator finds students whose marks are greater than 85.

```
> db.students.find({marks: {$lt: 60}})
< {
    _id: ObjectId('6aae9ea78f0f98eb52df222d'),
    roll: 18,
    name: 'Jai',
    age: 18,
    marks: 58,
    city: 'Gonda'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df223c'),
    roll: 33,
    name: 'Jai',
    age: 20,
    marks: 55,
    city: 'Ayodhya'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df224a'),
    roll: 47,
    name: 'Aman',
    age: 18,
    marks: 59,
    city: 'Gonda'
  }
> db.students.find({marks: {$eq: 88}})
< {
    _id: ObjectId('6aae9ea78f0f98eb52df221c'),
    roll: 1,
    name: 'Anamika',
    age: 19,
    marks: 88,
    city: 'Lucknow'
```

Screenshot 7: MongoDB query/result for marks greater than 85.

7. LESS THAN — $lt

The $lt comparison operator finds students whose marks are less than 60.

```
> db.students.find({marks: {$gte: 90}})
< {
    _id: ObjectId('6aae9ea78f0f98eb52df221e'),
    roll: 3,
    name: 'Vaishnavi',
    age: 19,
    marks: 92,
    city: 'Kanpur'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df2223'),
    roll: 8,
    name: 'Pratik',
    age: 19,
    marks: 90,
    city: 'Lucknow'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df2227'),
    roll: 12,
    name: 'Shreya',
    age: 19,
    marks: 95,
    city: 'Lucknow'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df222e'),
    roll: 19,
    name: 'Harshita',
    age: 20,
    marks: 91,
    city: 'Lucknow'
  }
```

Screenshot 8: MongoDB query/result for marks less than 60.

8. GREATER THAN OR EQUAL TO — $gte

The $gte comparison operator finds students whose marks are 90 or above.

```
> db.students.find({marks: {$lte: 70}})
< {
    _id: ObjectId('6aae9ea78f0f98eb52df221f'),
    roll: 4,
    name: 'Annu',
    age: 18,
    marks: 69,
    city: 'Ayodhya'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df2225'),
    roll: 10,
    name: 'Vivek',
    age: 18,
    marks: 65,
    city: 'Kanpur'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df222d'),
    roll: 18,
    name: 'Jai',
    age: 18,
    marks: 58,
    city: 'Gonda'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df222f'),
    roll: 20,
    name: 'Omkar',
    age: 19,
    marks: 63,
    city: 'Bareilly'
  }
```

Screenshot 9: MongoDB query/result for marks greater than or equal to 90.

9. LESS THAN OR EQUAL TO — $lte

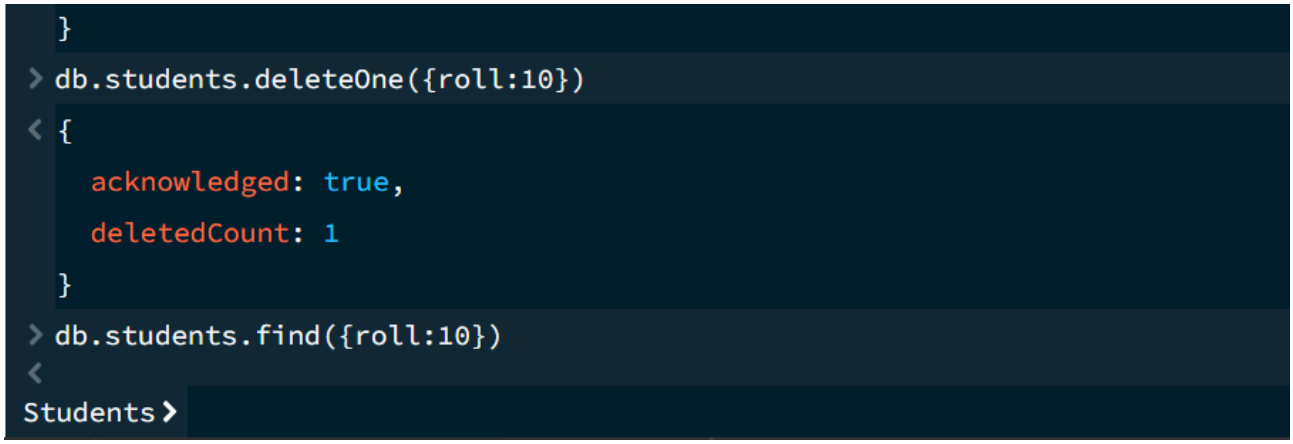
The $lte comparison operator finds students whose marks are 70 or below.

```
>_MONGOSH
}
db.students.updateOne(
  {roll:5},
  {$set:{marks:95}}
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
db.students.find({roll:5})
{
  _id: ObjectId('6aae9ea78f0f98eb52df2220'),
  roll: 5,
  name: 'Vansh',
  age: 19,
  marks: 95,
  city: 'Varanasi'
}
```

Screenshot 10: MongoDB query/result for marks less than or equal to 70.

10. UPDATE OPERATION

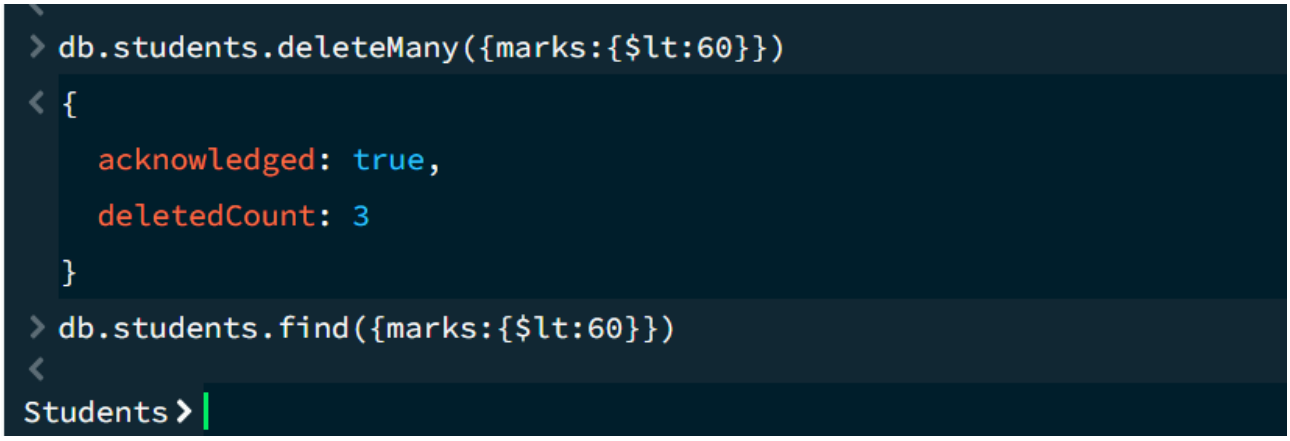
The update operation changes the marks of the student with roll number 5 to 95.



Screenshot 11: MongoDB update operation and resulting session.

11. DELETE ONE OPERATION

The delete-one operation removes the document matching roll number 10.



Screenshot 12: MongoDB delete-one operation and result.

12. DELETE MANY OPERATION

The delete-many operation removes students having marks below 60.

```
>_MONGOSH
> db.students.find({
    $and:[
      {marks:{$gt:80}},{age:19}
    ]
  })
< {
    _id: ObjectId('6aae9ea78f0f98eb52df221c'),
    roll: 1,
    name: 'Anamika',
    age: 19,
    marks: 88,
    city: 'Lucknow'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df221e'),
    roll: 3,
    name: 'Vaishnavi',
    age: 19,
    marks: 92,
    city: 'Kanpur'
  }
  {
```

Screenshot 13: MongoDB delete-many operation and the related query session.

13. AND OPERATION — $and

The $and logical operator requires both conditions to be satisfied: marks greater than 80 and age equal to 19.

```
>_MONGOSH
> db.students.find({
    $or:[
      {city:"Lucknow"},{city:"Kanpur"}
    ]
  })
{
  _id: ObjectId('6aae9ea78f0f98eb52df221c'),
  roll: 1,
  name: 'Anamika',
  age: 19,
  marks: 88,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df221e'),
  roll: 3,
  name: 'Vaishnavi',
  age: 19,
  marks: 92,
  city: 'Kanpur'
}
{
```

Screenshot 14: MongoDB AND condition.

14. OR OPERATION — $or

The $or logical operator returns students whose city is either Lucknow or Kanpur.

```
>_MONGOSH
> db.students.find().sort({marks:1})
< {
    _id: ObjectId('6aae9ea78f0f98eb52df2237'),
    roll: 28,
    name: 'Bhaskar',
    age: 18,
    marks: 61,
    city: 'Gorakhpur'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df2246'),
    roll: 43,
    name: 'Sarika',
    age: 19,
    marks: 62,
    city: 'Ayodhya'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df222f'),
    roll: 20,
    name: 'Omkar',
    age: 19,
    marks: 63,
    city: 'Bareilly'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df224c'),
    roll: 49,
    name: 'Rahul',
    age: 19,
    marks: 64,
    city: 'Bareilly'
  }
```

Screenshot 15: MongoDB OR condition.

15. SORT OPERATION — ASCENDING

The sort operation orders the student records by marks from lowest to highest. Ascending order is represented by 1.

```
>_MONGOSH
> db.students.find().sort({marks:-1})
< {
    _id: ObjectId('6aae9ea78f0f98eb52df223e'),
    roll: 35,
    name: 'Krishna',
    age: 19,
    marks: 96,
    city: 'Prayagraj'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df2227'),
    roll: 12,
    name: 'Shreya',
    age: 19,
    marks: 95,
    city: 'Lucknow'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df2220'),
    roll: 5,
    name: 'Vansh',
    age: 19,
    marks: 95,
    city: 'Varanasi'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df2235'),
    roll: 26,
    name: 'Anushka',
    age: 20,
    marks: 94,
    city: 'Lucknow'
  }
```

Screenshot 16: MongoDB ascending sort by marks.

15. SORT OPERATION — DESCENDING

The descending sort orders the student records from highest marks to lowest marks. Descending order is represented by -1.

```
>_MONGOSH
> db.students.find().limit(5)
< {
    _id: ObjectId('6aae9ea78f0f98eb52df221c'),
    roll: 1,
    name: 'Anamika',
    age: 19,
    marks: 88,
    city: 'Lucknow'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df221d'),
    roll: 2,
    name: 'Uday',
    age: 18,
    marks: 76,
    city: 'Jaunpur'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df221e'),
    roll: 3,
    name: 'Vaishnavi',
    age: 19,
```

Screenshot 17: MongoDB descending sort by marks.

16. LIMIT OPERATION

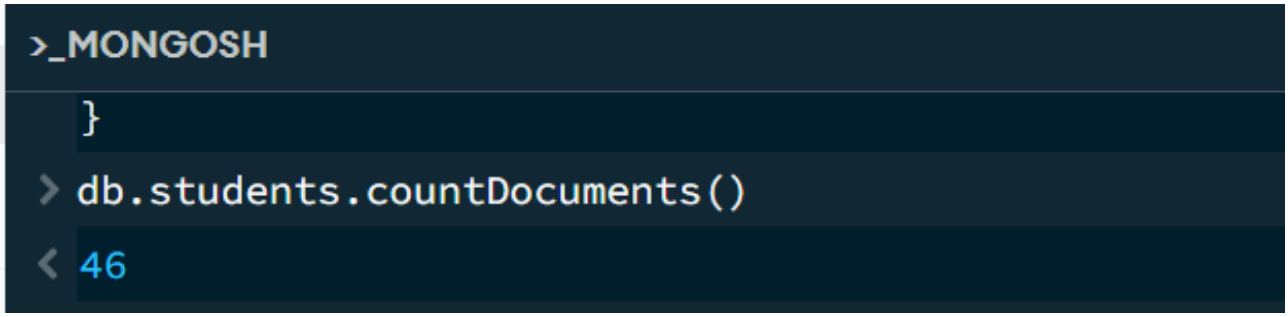
The limit operation displays only the first five documents from the query result.

```
>_MONGOSH
> db.students.find({city:"Lucknow"})
< {
    _id: ObjectId('6aae9ea78f0f98eb52df221c'),
    roll: 1,
    name: 'Anamika',
    age: 19,
    marks: 88,
    city: 'Lucknow'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df2221'),
    roll: 6,
    name: 'Amrita',
    age: 20,
    marks: 85,
    city: 'Lucknow'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df2223'),
    roll: 8,
    name: 'Pratik',
    age: 19,
```

Screenshot 18: MongoDB limit operation.

17. SEARCH BY CITY

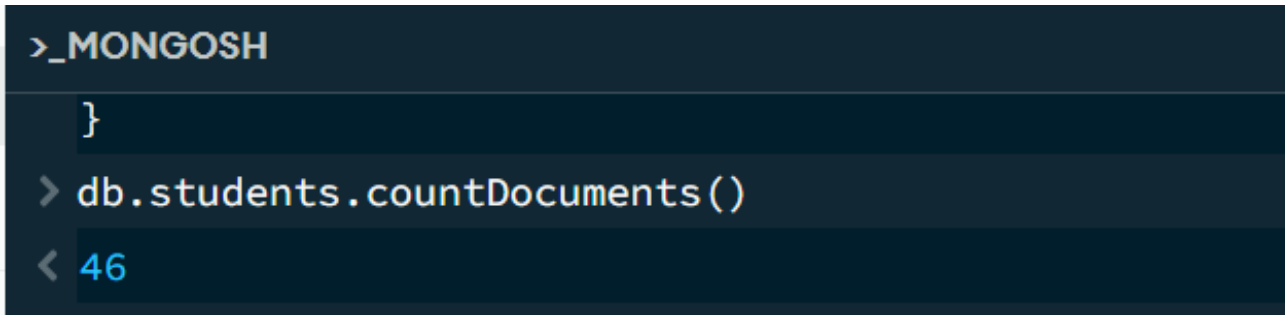
The city search retrieves all students whose city is Lucknow.



Screenshot 19: MongoDB session showing the city search context.

18. COUNT DOCUMENTS

The count operation counts the documents currently present in the collection after the delete operations. The reference screenshot shows the resulting count as 46.



Screenshot 20: MongoDB document count after delete operations.

MONGODB OPERATORS USED

Operator / Operation	Use in the project
$eq	Finds values exactly equal to a specified value.
$gt	Finds values greater than a specified value.
$lt	Finds values less than a specified value.
$gte	Finds values greater than or equal to a specified value.
$lte	Finds values less than or equal to a specified value.
$and	Requires all specified conditions to be true.
$or	Returns documents when at least one specified condition is true.
sort()	Arranges query results in ascending or descending order.
limit()	Restricts the number of returned documents.
updateOne()	Updates one matching document.
deleteOne()	Deletes one matching document.
deleteMany()	Deletes all documents matching a condition.
countDocuments()	Counts documents currently in the collection.

RESULT SUMMARY

The reference project demonstrates a complete basic workflow for managing student records in MongoDB: creating/selecting a database, working with a student collection, inserting 50 records, retrieving and filtering data, updating and deleting documents, sorting and limiting results, searching by city, and counting the remaining documents.

According to the supplied reference screenshot, the final document count shown after the delete operations is 46.

CONCLUSION

This project provided practical knowledge of NoSQL databases using MongoDB. A Student Management System containing student information was created and different MongoDB operations were performed on the data.

Through this project, I learned how to insert, retrieve, update and delete documents. I also learned how ascending and descending ordering works, how comparison operators such as $eq, $gt, $lt, $gte and $lte can be used, and how logical operators such as $and and $or help combine conditions.

Overall, this project helped me understand how MongoDB can be used to manage and query student data efficiently.

PROJECT SUBMISSION DETAILS

Name	Jijivisha Tiwari
Roll No.	66
Class	BCA DS & AI 23
Teacher	Mr. Harendra Singh Rajpoot
Academic Session	2026–2027

END OF PROJECT REPORT